CHAPTER 5

Testing in Flask

Something that is untested is broken.

This quote comes from an unknown source; however, it's not entirely true but most of it is right. Untested applications are always an unsafe bet to make. While the developers are confident about their work, in real world things work out differently; hence it's always a good idea to test the application throughout. Untested applications also make it hard to improve the existing code. However with automated tests, it's always easy to make changes and instantly know when something breaks. So testing not just only ensures if the application is behaving the way it is expected to, it also facilitates continuous development.

This chapter covers automated unit testing of REST APIs, and before we get into the actual implementation, we'll look into what unit testing is and the principles behind.

Introduction

Most software developers out there are usually already familiar with the term "unit testing," but for those who are not, unit testing revolves around the concept of breaking a large set of code into individual units to be tested in isolation. So typically in such a case, a larger set of code is software, and individual components are the units to be tested in isolation. Thus in our case, a single API request is a unit to be tested. Unit testing is the first level of software development and is usually done by software developers.

© Kunal Relan 2019
K. Relan, *Building REST APIs with Flask*, https://doi.org/10.1007/978-1-4842-5022-8_5

Let's look into some benefits of unit testing:

1. Unit tests are simple tests for a very narrow block of code, serving as a building block of the bigger spectrum of the application testing.
2. Being narrowly scoped, unit tests are the easiest to write and implement.
3. Unit tests increase confidence in modifying the code and are also the first point of failure if implemented correctly prompting the developer about parts of logic breaking the application.
4. Writing unit tests makes the development process faster, since it makes developers to do less of fuzzy testing and helps them catch the bugs sooner.
5. Catching and fixing bugs in development using unit tests is easier and less expensive than doing it after the code is deployed in production.
6. Unit tests are also a more reliable way of testing in contrast to manual fuzz tests.

Setting Up Unit Tests

So, in this section, we'll jump right into the action and start on implementing the tests; for the same we'll use a library called unittest2 which is an extension to the original unit testing framework of Python called unittest.

Let's go ahead and install the library first.

```
(venv)$ pip install unittest2
```

This shall install unittest2 for us; next we'll set up a base test class that we'll import in all our test files. This base class will set up the base for the tests and initiate the test client as the name suggests. So go ahead and create a file called test_base.py in utils folder.

Now let's configure our testing environment, so open up your config.py and add the following code to add testing config.

```
class TestingConfig(Config):
    TESTING = True
    SQLALCHEMY_ECHO = False
    JWT_SECRET_KEY = 'JWT-SECRET'
    SECRET_KEY= 'SECRET-KEY'
    SECURITY_PASSWORD_SALT= 'PASSWORD-SALT'
    MAIL_DEFAULT_SENDER= '
    MAIL_SERVER= 'smtp.gmail.com'
    MAIL_PORT= 465
    MAIL_USERNAME= "
    MAIL_PASSWORD= "
    MAIL_USE_TLS= False
    MAIL_USE_SSL= True
    UPLOAD_FOLDER= 'images'
```

Notice that we won't configure the SQLAlchemy URI here, which we'll do in test_base.py

Next, add the following lines to import the required dependencies in test_base.py

```
import unittest2 as unittest
from main import create_app
from api.utils.database import db
from api.config.config import TestingConfig
import tempfile
```

Next add the BaseTestCase class with the following code.

```
class BaseTestCase(unittest.TestCase):
    """A base test case"""
    def setUp(self):
        app = create_app(TestingConfig)
        self.test_db_file = tempfile.mkstemp()[1]
        app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///' +
        self.test_db_file
        with app.app_context():
            db.create_all()
        app.app_context().push()
        self.app = app.test_client()

    def tearDown(self):
        db.session.close_all()
        db.drop_all()
```

Here we are creating the SQLAlchemy sqlite database on the fly using tempfile.

What we just created previously is called a stub, which is a module that acts as a temporary replacement for a called module providing the same output as the actual product.

So the preceding method will run before every test is run and it spawns a new test client. We'll import this method in all the tests we create. A test is recognized by all the methods in the class which starts with test_ prefix. Here we'll have a unique database URL every time since we have configured tempfile, and we'll postfix it with a timestamp and then we have configured TESTING= True in app config which will disable error catching to enable better testing, and then finally we run db.create_all() to create the DB tables for the application.

Next we have defined another method tearDown which will remove the current database file and use a fresh database file for every test.

Unit Testing User Endpoints

So now we'll start writing the tests, and the first step to it is by creating a folder called tests in api directory where we'll create all our test files. So go ahead and create tests folder and create our first test file called test_users.py.

Now add the following imports in test_users.py

```
import json
from api.utils.test_base import BaseTestCase
from api.models.users import User
from datetime import datetime
import unittest2 as unittest
from api.utils.token import generate_verification_token,
confirm_verification_token
```

Once done, we'll define another method to create users using the SQLAlchemy model to facilitate testing.

Add this to the file next.

```
def create_users():
    user1 = User(email="kunal.relan12@gmail.com",
    username='kunalrelan12',
    password=User.generate_hash('helloworld'),
    isVerified=True).create()
    user2 = User(email="kunal.relan123@gmail.com",
    username='kunalrelan125',
    password=User.generate_hash('helloworld')).create()
```

Now we have our imports and the method to create users; next we'll define TestUsers class to hold all our tests.

```
class TestUsers(BaseTestCase):
    def setUp(self):
        super(TestUsers, self).setUp()
```

```
        create_users()

if __name__ == '__main__':
    unittest.main()
```

Add this code to the file which will import our base test class and set up the test client and call create_users() method to create the users. Notice that in create_users() method, we have created one verified and one unverified user so that we can cover up all the test cases. Now we can start writing our unit tests. Add the following code inside TestUsers() class.

We'll start by testing the login endpoint, and since we just created a verified user, we should be allowed to log in with a valid set of credentials.

```
    def test_login_user(self):
        user = {
          "email" : "kunal.relan12@gmail.com",
          "password" : "helloworld"
        }
        response = self.app.post(
            '/api/users/login',
            data=json.dumps(user),
            content_type='application/json'
        )
        data = json.loads(response.data)
        self.assertEqual(200, response.status_code)
        self.assertTrue('access_token' in data)
```

Add the following code inside the TestUsers class, and we should have our first unit test in which we create a user object and post the user to login endpoint. Once we receive the response, we'll use assertion to check if we got the expected status code and access_token in the response. An assertion is a boolean expression which will be true unless there is a bug or the conditional statement doesn't match. Unit test provides a list of assertion methods we can use to validate our tests.

But assertEqual(), assertNotEqual(), assertTrue(), and assertNotTrue() cover most of it.

Here assertEqual() and assertNotEqual() match for values, and assertTrue() and assertNotTrue() check if the value of passed variable being a boolean.

Now let's run our first test, so just open your terminal and activate your virtual environment.

In your terminal run the following command to run the tests.

```
(venv)$ python -m unittest discover api/tests
```

The preceding command will run all the test files inside the tests directory; since we have only one test for now, we can see the result of our tests in the following figure.

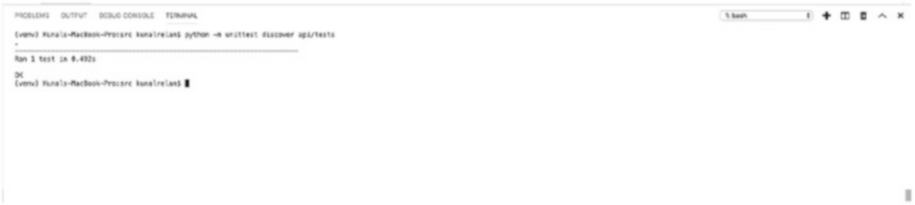

Figure 5-1. *Running unit tests*

So this was one way of running our unit tests, and before we process further with writing more tests, I'd like to introduce you to another extension to unittest library called nose which makes testing easier, so let's go ahead and install nose.

Use the following code to install nose.

```
(venv)$ pip install nose
```

And now once we have nose, let's see how we can use nose to run our tests since moving on we'll use nose to run all our tests.

By default nose will find all the test files using a (?:\b|_)[Tt]est regular expression; however, you can also specify the filename to test. Let's run the same test again by using nose.

```
(venv)$ nosetests
```

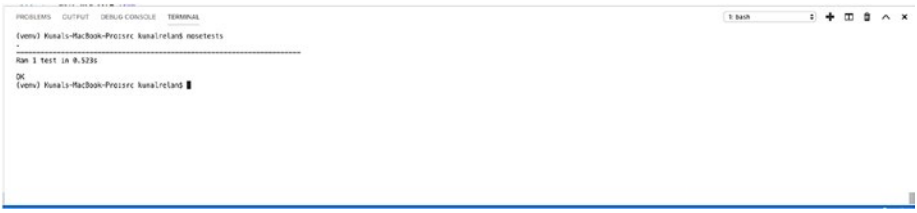

Figure 5-2. *Running unit tests with nose*

As you can see in the preceding figure, we can run our tests using a simple nosetest command. Next let's write unit tests for user model again.

So our goal here is to cover all the scenarios and check the application behavior in each of the scenarios; next we'll test login API when the user is not verified and when wrong credentials are submitted.

Add the following code for the respective tests.

```
def test_login_user_wrong_credentials(self):
    user = {
      "email" : "kunal.relan12@gmail.com",
      "password" : "helloworld12"
    }
    response = self.app.post(
        '/api/users/login',
        data=json.dumps(user),
        content_type='application/json'
    )
    data = json.loads(response.data)
    self.assertEqual(401, response.status_code)
```

```
def test_login_unverified_user(self):
    user = {
      "email" : "kunal.relan123@gmail.com",
      "password" : "helloworld"
    }
    response = self.app.post(
        '/api/users/login',
        data=json.dumps(user),
        content_type='application/json'
    )
    data = json.loads(response.data)
    self.assertEqual(400, response.status_code)
```

In the preceding code, in test_login_user_wrong_credentials method, we check for 401 status code in the response as we are supplying wrong credentials, and in test_login_unverified_user() method, we are trying to login with an unverified user which shall throw 400 error.

Next let's test the create_user endpoint and start by creating a test to create a user with correct fields to create a new user.

```
def test_create_user(self):
    user = {
      "username" : "kunalrelan2",
      "password" : "helloworld",
      "email" : "kunal.relan12@hotmail.com"
    }

    response = self.app.post(
        '/api/users/',
        data=json.dumps(user),
        content_type='application/json'
    )
```

```
        data = json.loads(response.data)
        self.assertEqual(201, response.status_code)
        self.assertTrue('success' in data['code'])
```

The preceding code will request the Create user endpoint with a new user object and shall be able to do so and respond with a 201 status code.

Next we'll add another test when username is not supplied to the Create user endpoint, and in this case, we shall get a 422 response. Here is the code for that.

```
    def test_create_user_without_username(self):
        user = {
          "password" : "helloworld",
          "email" : "kunal.relan12@hotmail.com"
        }

        response = self.app.post(
            '/api/users/',
            data=json.dumps(user),
            content_type='application/json'
        )
        data = json.loads(response.data)
        self.assertEqual(422, response.status_code)
```

Now we can move on to testing our confirm email endpoint, and here we'll first create a unit test with valid email, so you noticed we had an unverified user created in create_users() method, and here first we'll generate a validation token since we are not reading the email using the unit tests and then send the token to confirm email endpoint.

```
    def test_confirm_email(self):
        token = generate_verification_token('kunal.relan123@
        gmail.com')
```

```
        response = self.app.get(
            '/api/users/confirm/'+token
        )
        data = json.loads(response.data)
        self.assertEqual(200, response.status_code)
        self.assertTrue('success' in data['code'])
```

Next, we'll write another test with email of an already verified user to test if we get 422 in response status code.

```
    def test_confirm_email_for_verified_user(self):
        token = generate_verification_token('kunal.relan12@
        gmail.com')

        response = self.app.get(
            '/api/users/confirm/'+token
        )
        data = json.loads(response.data)
        self.assertEqual(422, response.status_code)
```

And the last one for this endpoint is we'll supply an incorrect email and should get a 404 response status code.

```
    def test_confirm_email_with_incorrect_email(self):
        token = generate_verification_token('kunal.relan43@
        gmail.com')

        response = self.app.get(
            '/api/users/confirm/'+token
        )
        data = json.loads(response.data)
        self.assertEqual(404, response.status_code)
```

Once we have our tests in place, it's time to test them all, so go ahead and use nosetests and run the tests.

Figure 5-3. *Nosetests on test_users.py*

So these are all the tests we want to cover with user model; next we can move on to authors and books.

Next let's create test_authors.py and we'll add the dependencies with a few changes, so add the following lines to import the required dependencies.

```
import json
from api.utils.test_base import BaseTestCase
from api.models.authors import Author
from api.models.books import Book
from datetime import datetime
from flask_jwt_extended import create_access_token
import unittest2 as unittest
import io
```

Next we'll define two helper methods, namely, create_authors and login, and add the following code for the same.

```
def create_authors():
    author1 = Author(first_name="John", last_name="Doe").create()
    author2 = Author(first_name="Jane", last_name="Doe").create()
```

We'll create two authors for the test using the method defined previously, and login method will generate a login token and return for authorized only routes.

```
def login():
    access_token = create_access_token(identity = 'kunal.relan@
    hotmail.com')
    return access_token
```

Next let's define our test class like we did earlier and initiate it.

```
class TestAuthors(BaseTestCase):
    def setUp(self):
        super(TestAuthors, self).setUp()
        create_authors()

if __name__ == '__main__':
    unittest.main()
```

Now we have the base of our author unit tests, and we can add the following test cases which should be self-explanatory.

Here we'll create a new author using POST author endpoint with the JWT token we generate using login method and expect author object with 201 status code in response.

```
    def test_create_author(self):
        token = login()
        author = {
            'first_name': 'Johny',
            'last_name': 'Doee'
        }
        response = self.app.post(
            '/api/authors/',
            data=json.dumps(author),
            content_type='application/json',
            headers= { 'Authorization': 'Bearer '+token }
        )
```

```
        data = json.loads(response.data)
        self.assertEqual(201, response.status_code)
        self.assertTrue('author' in data)
```

Here we'll try creating an author with authorization header, and it should return 401 in the response status code.

```
    def test_create_author_no_authorization(self):
        author = {
            'first_name': 'Johny',
            'last_name': 'Doee'
        }

        response = self.app.post(
            '/api/authors/',
            data=json.dumps(author),
            content_type='application/json',
        )
        data = json.loads(response.data)
        self.assertEqual(401, response.status_code)
```

In this test case, we'll try creating an author without last_name field, and it should respond back with 422 status code.

```
    def test_create_author_no_name(self):
        token = login()
        author = {
            'first_name': 'Johny'
        }

        response = self.app.post(
            '/api/authors/',
            data=json.dumps(author),
            content_type='application/json',
```

```
            headers= { 'Authorization': 'Bearer '+token }
        )
        data = json.loads(response.data)
        self.assertEqual(422, response.status_code)
```

In this one we'll test upload avatar endpoint and use io to create a temp image file and send it as multipart/form-data to upload the image.

```
    def test_upload_avatar(self):
        token = login()
        response = self.app.post(
            '/api/authors/avatar/2',
           data=dict(avatar=(io.BytesIO(b'test'),
           'test_file.jpg')),
            content_type='multipart/form-data',
            headers= { 'Authorization': 'Bearer '+ token }
        )
        self.assertEqual(200, response.status_code)
```

Here, we'll test the upload avatar by supplying a CSV file instead, and as expected it should not respond with 200 status code.

```
    def test_upload_avatar_with_csv_file(self):
        token = login()
        response = self.app.post(
            '/api/authors/avatar/2',
           data=dict(file=(io.BytesIO(b'test'), 'test_file.csv')),
            content_type='multipart/form-data',
            headers= { 'Authorization': 'Bearer '+ token }
        )
        self.assertEqual(422, response.status_code)
```

In this test, we'll get all the authors using GET all authors endpoint.

```
def test_get_authors(self):
    response = self.app.get(
        '/api/authors/',
        content_type='application/json'
    )
    data = json.loads(response.data)
    self.assertEqual(200, response.status_code)
    self.assertTrue('authors' in data)
```

Here we have a unit test for GET author by ID endpoint, and it'll return 200 response status code and author object.

```
def test_get_author_detail(self):
    response = self.app.get(
        '/api/authors/2',
        content_type='application/json'
        )
    data = json.loads(response.data)
    self.assertEqual(200, response.status_code)
    self.assertTrue('author' in data)
```

In this test we'll update the author object on the recently created author, and it shall also return 200 status code in the response.

```
def test_update_author(self):
    token = login()
    author = {
        'first_name': 'Joseph'
    }
    response = self.app.put(
        '/api/authors/2',
        data=json.dumps(author),
```

```
            content_type='application/json',
            headers= { 'Authorization': 'Bearer '+token }
        )
        self.assertEqual(200, response.status_code)
```

In this test we'll delete author object and expect 204 response status code.

```
    def test_delete_author(self):
        token = login()
        response = self.app.delete(
            '/api/authors/2',
            headers= { 'Authorization': 'Bearer '+token }
        )
        self.assertEqual(204, response.status_code)
```

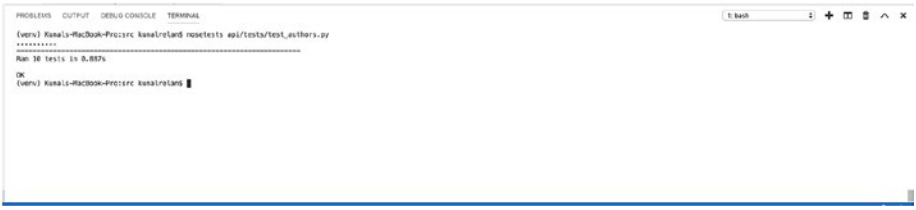

Figure 5-4. *Authors test*

So now you can run authors test like in the previous figure, and it should all pass like in that figure; next we'll move to books model test.

For books model tests, we can modify the author tests and set up unit tests for books in the same module, so let's update create_authors method to create some books as well; just go ahead and update the method with following code.

```
def create_authors():
    author1 = Author(first_name="John", last_name="Doe").
    create()
```

```
Book(title="Test Book 1", year=datetime(1976, 1, 1),
author_id=author1.id).create()
Book(title="Test Book 2", year=datetime(1992, 12, 1),
author_id=author1.id).create()

author2 = Author(first_name="Jane", last_name="Doe").
create()
Book(title="Test Book 3", year=datetime(1986, 1, 3),
author_id=author2.id).create()
Book(title="Test Book 4", year=datetime(1992, 12, 1),
author_id=author2.id).create()
```

And then here are the unit tests for book routes.

```
def test_create_book(self):
    token = login()
    author = {
        'title': 'Alice in wonderland',
        'year': 1982,
        'author_id': 2
    }

    response = self.app.post(
        '/api/books/',
        data=json.dumps(author),
        content_type='application/json',
        headers= { 'Authorization': 'Bearer '+token }
    )
    data = json.loads(response.data)
    self.assertEqual(201, response.status_code)
    self.assertTrue('book' in data)

def test_create_book_no_author(self):
    token = login()
```

```
        author = {
            'title': 'Alice in wonderland',
            'year': 1982
        }

        response = self.app.post(
            '/api/books/',
            data=json.dumps(author),
            content_type='application/json',
            headers= { 'Authorization': 'Bearer '+token }
        )
        data = json.loads(response.data)
        self.assertEqual(422, response.status_code)

    def test_create_book_no_authorization(self):
        author = {
            'title': 'Alice in wonderland',
            'year': 1982,
            'author_id': 2
        }

        response = self.app.post(
            '/api/books/',
            data=json.dumps(author),
            content_type='application/json'
        )
        data = json.loads(response.data)
        self.assertEqual(401, response.status_code)

    def test_get_books(self):
        response = self.app.get(
            '/api/books/',
            content_type='application/json'
        )
```

```
        data = json.loads(response.data)
        self.assertEqual(200, response.status_code)
        self.assertTrue('books' in data)

    def test_get_book_details(self):
        response = self.app.get(
            '/api/books/2',
            content_type='application/json'
            )
        data = json.loads(response.data)
        self.assertEqual(200, response.status_code)
        self.assertTrue('books' in data)

    def test_update_book(self):
        token = login()
        author = {
            'year': 1992,
            'title': 'Alice'
        }
        response = self.app.put(
            '/api/books/2',
            data=json.dumps(author),
            content_type='application/json',
            headers= { 'Authorization': 'Bearer '+token }
        )
        self.assertEqual(200, response.status_code)

    def test_delete_book(self):
        token = login()
        response = self.app.delete(
            '/api/books/2',
```